1 Initial Works

1.1 Docker Compose Setup

先来看这份 `docker-compose.yml` 文件：

```yaml
services:
  db:
    image: mysql:latest
    container_name: mysql_server
    restart: always
    command: --innodb_redo_log_capacity=1073741824
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: ${MYSQL_DATABASE}
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql
      - ./ml-latest:/var/lib/mysql-files/ml-latest
      - ./personality-isf2018:/var/lib/mysql-files/personality-isf2018

  app:
    build: .
    container_name: python_app
    env_file:
      - .env
    ports:
      - "127.0.0.1:5001:5000"
    depends_on:
      - db

  web:
    image: nginx:latest
    container_name: nginx_server
    ports:
      - "80:80"
    volumes:
      - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - app

volumes:
  mysql_data:
```

现在的Docker已经很方便了，你只要在官网下载了Docker Desktop，安装好之后就可以直接在命令行里使用 `docker compose` 了。这份文件的 `services` 部分定义了三个服务： `db` （MySQL数据库）， `app` （Python应用），和 `web` （Nginx服务器）。每个服务都有自己的配置，包括环境变量、端口映射和依赖关系。

db

这段代码定义了名为 `mysql_server` 的服务。

`image: mysql:latest` 表示使用最新的MySQL镜像。

通过 `ports: - "3306:3306"` 的配置，MySQL服务器的3306端口被映射到主机的3306端口，这样你就可以通过 `localhost:3306` 来访问数据库了。

然后，通过 `./init.sql:/docker-entrypoint-initdb.d/init.sql` `./ml-latest:/var/lib/mysql-files/ml-latest` 和 `./personality-isf2018:/var/lib/mysql-files/personality-isf2018` 这三行代码，将 `init.sql` 文件挂载到MySQL初始化目录，并且将 `ml-latest` 和 `personality-isf2018` 这两个目录挂载到容器内的指定路径，这样你就可以在数据库中访问这些数据了。

app

这段代码定义了名为 `python_app` 的服务。

它指定了为 `127.0.0.1:5001:5000` 的端口映射，这样你就可以通过 `localhost:5001` 来访问容器内的 `5000` 端口了。

其通过 `depends_on: - db` 指定了它依赖于 `db` 服务，这意味着在启动 `app` 服务之前，Docker会先启动 `db` 服务。

web

这段代码定义了名为 `nginx_server` 的服务。

`image: nginx:latest` 表示使用最新的Nginx镜像。

通过 `./nginx.conf:/etc/nginx/conf.d/default.conf:ro` 这行代码，将本地的 `nginx.conf` 文件 **以只读方式** 挂载到容器内的Nginx配置目录，其中是我们写的关于Nginx将请求发给 `app` 的配置。

1.2 init.sql and ER Diagram

我们以这段代码为例

```
DROP TABLE IF EXISTS init_run_log;
CREATE TABLE init_run_log (
    id INT AUTO_INCREMENT PRIMARY KEY,
    stage VARCHAR(64) NOT NULL,
    ran_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO init_run_log(stage) VALUES ('started');

CREATE TABLE IF NOT EXISTS movie (
    movieId INT,
    title VARCHAR(255) NOT NULL,
    runtimeMinutes INT,
    PRIMARY KEY (movieId)
) CHARACTER SET utf8mb4;

CREATE TABLE IF NOT EXISTS movie_genre (
    movieId INT,
    genre VARCHAR(100) NOT NULL,
    PRIMARY KEY (movieId, genre)
);

INSERT INTO init_run_log(stage) VALUES ('tables_created');

LOAD DATA INFILE '/var/lib/mysql-files/ml-latest/movies.csv'
IGNORE INTO TABLE movie
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(movieId, title, @runtimeMinutes)
SET runtimeMinutes = NULLIF(TRIM(@runtimeMinutes), '');

LOAD DATA INFILE '/var/lib/mysql-files/ml-latest/movie_genres.csv'
IGNORE INTO TABLE movie_genre
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;

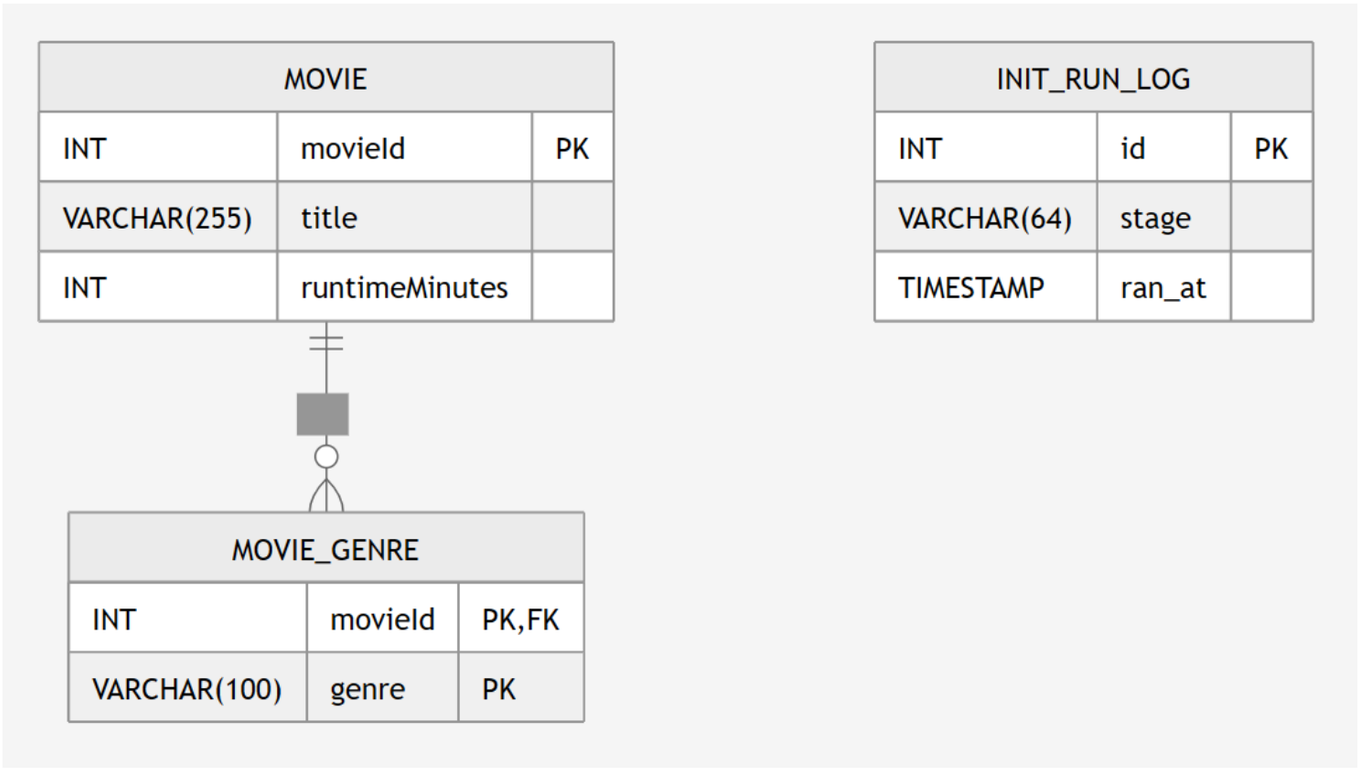
INSERT INTO init_run_log(stage) VALUES ('loaded');

CREATE INDEX idx_movie_title ON movie(title);

ALTER TABLE movie_genre
    ADD CONSTRAINT fk_mg_movie FOREIGN KEY (movieId)
    REFERENCES movie(movieId) ON DELETE CASCADE;

INSERT INTO init_run_log(stage) VALUES ('finished');
```

使用ER Diagram会有利于理解数据库结构，甚至有助于设计。

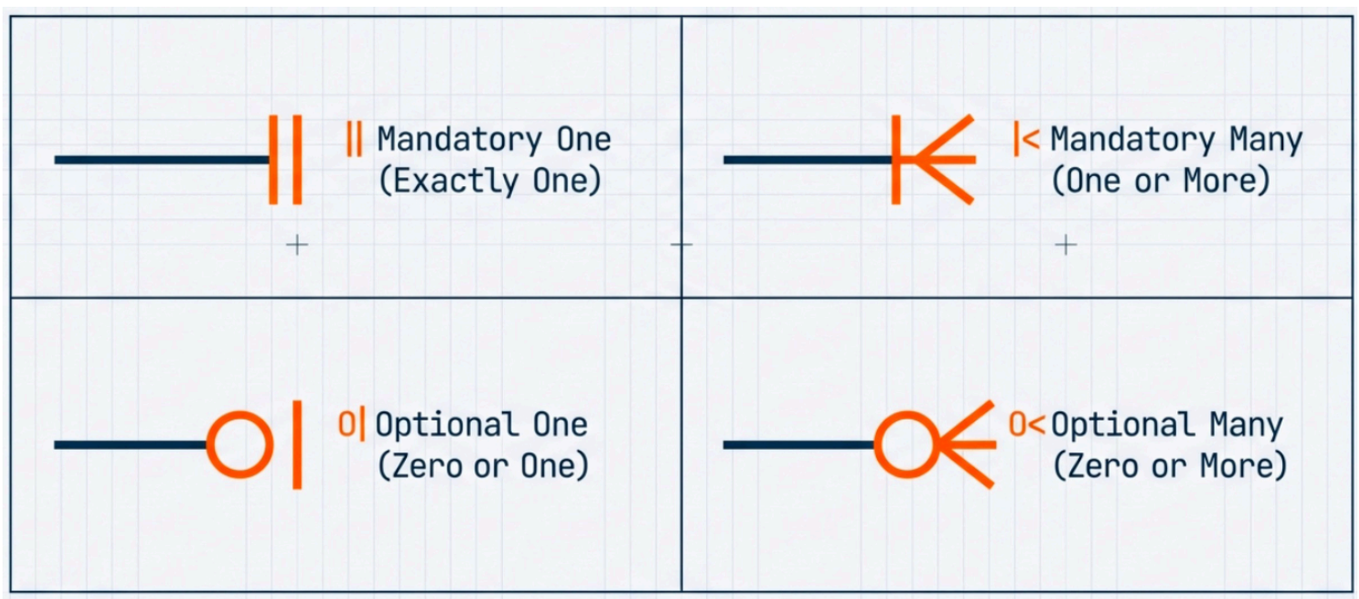


图中的PK就是我们在 `PRIMARY KEY (movieId)` 等地方定义的主键，其中对于 `movie_genre` 来说，主键是由 `movieId` 和 `genre` 这两列组成的复合主键。这也就是说，一部电影可以有多个类型，但同一部电影的同一类型只能出现一次。

而FK就是在代码结尾定义的外键。它表示 `movie_genre` 表中的 `movieId` 列必须对应 `movie` 表中的一个有效的 `movieId` ，并且如果 `movie` 表中的某个电影被删除了，那么与之相关的 `movie_genre` 表中的记录也会被自动删除（因为我们使用了 `ON DELETE CASCADE` ）。

在命名表时也有讲究，通常我们会使用单数形式的名词来命名表（如 `movie` 而不是 `movies` ）。

而线两头的脚也有含义，它表示对应关系。在这个例子中， `movie` 和 `movie_genre` 之间的关系是“一对多”的关系，因为一部电影可以有多个类型，但每个类型只能对应一部电影。同时我们认可一部电影可以没有类型，所以我们在这张图上画了一个圈。所有的表示方法如下图所示：



这样的ER Diagram可以帮我们整理数据之间的关系，尤其是，当出现“多对多”的关系时，我们就需要引入一个新的表来打破这个关系，因为关系数据库不支持直接的多对多关系。比如，如果我们有一个 `student` 表和一个 `course` 表，并且一个学生可以选修多门课程，而一门课程也可以被多个学生选修，那么我们就需要引入一个新的表（比如 `enrollment` ）来表示学生和课程之间的关系。

1.3 Data Cleaning

让我们以这一段代码为例

```
CREATE TABLE IF NOT EXISTS personality_data_stage (
    userId VARCHAR(64),
    openness DECIMAL(5,2),
    is_personalized INT
) CHARACTER SET utf8mb4;

LOAD DATA INFILE '/var/lib/the/path/to/file.csv'
INTO TABLE personality_data_stage
FIELDS TERMINATED BY ','
OPTIONALLY ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(@u, @o, @junk1, @junk2, @ip)
SET userId = NULLIF(TRIM(@u),''),
    openness = NULLIF(TRIM(@o),''),
    is_personalized = NULLIF(TRIM(@ip),'');

CREATE TABLE IF NOT EXISTS personality_data (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    userId VARCHAR(64) NOT NULL UNIQUE,
    openness DECIMAL(5,2) NOT NULL,
    is_personalized INT NOT NULL
) CHARACTER SET utf8mb4;

INSERT INTO personality_data (userId, openness, is_personalized)
SELECT userId, openness, is_personalized
FROM personality_data_stage
WHERE userId IS NOT NULL
  AND openness IS NOT NULL
  AND is_personalized IS NOT NULL;
```

在这个例子中，我们从共有5列的CSV文件中只提取了3列（`userId`、`openness`和`is_personalized`）。而舍弃掉中间的两列。

在导入时，我们使用`TRIM`函数来去除字段值两端的空格，并使用`NULLIF`函数将空字符串转换为`NULL`。然后再开始将这些数据转移到正式表中。

在这里我们接触到了`SELECT`语句。它可以使用`WHERE`子句来过滤数据，只有满足条件的数据才会被插入到正式表中。例如在上面的例子中，我们只插入了那些`userId`、`openness`和`is_personalized`都不为`NULL`的记录。

对于缺失数据的样本，还有其他的处理方法。对于原始数据集较完整的情况，可以使用上面例子中的直接删除法。而假如数据不完整，可以使用如均值插补、中位数插补、众数插补等的补植方法。也可以选择保留原始数据集，但是新增一列`is_value_missing (True/False)`来标记数据是否缺失，便于后续灵活使用。

MySQL对于数值读取的容错率比较高，对于处理不当的原始数据（例如空格没去），通常都能正确地读取到。但是对于时间数据，就更加需要主动清洗。MySQL的默认时间格式是ISO标准的，也就是说，如果你使用如下定义

```
my_date DATE
```

那么MySQL就会期望你输入的时间数据是`YYYY-MM-DD`这样的格式。

那么假如我们知道输入集中还包含`DD/MM/YYYY`这样的时间格式，我们就可以使用`STR_TO_DATE`函数来尝试解析这些不同格式的时间数据，并使用`COALESCE`函数来选择第一个成功解析的结果。示例如下：

```
SET some_date = COALESCE(
    STR_TO_DATE(TRIM(@d), '%Y-%m-%d'),
    STR_TO_DATE(TRIM(@d), '%d/%m/%Y')
)
```

最后，数据清理不需要有洁癖，对于过于复杂的工作，放心大胆地使用Python吧！

2 SELECT Queries

子句	用法说明	示例
`SELECT`	用于指定需要查询的字段，可以查询一个或多个列。	`SELECT id, name`
`FROM`	用于指定查询的数据表。	`FROM employees`
`WHERE`	用于在分组前过滤满足条件的数据。	`WHERE salary >= 5000`
`GROUP BY`	用于将查询结果按某个字段分组，通常配合`COUNT`、`SUM`、`AVG`等聚合函数使用。	`GROUP BY department_id`
`HAVING`	用于在分组后对分组结果进行过滤。	`HAVING COUNT(*) > 3`
`ORDER BY`	用于对结果集进行排序，默认升序，可用`DESC`指定降序，`ASC`指定升序（默认）。	`ORDER BY salary DESC`

当然只是这样过于抽象了，实际情况也会更加复杂。我们来跟着例子逐步理解。

2.1 Basic Queries

```
SELECT staffNo,
ROUND(salary/12,2) AS 'Monthly Pay'
from Staff;
```

这里我们取出了 `salary` 并将其除以12，然后取两位小数，最后将其命名为“月薪”。

哦对了，相信你可能对SQL中关键字的大小写使用可能有疑问。答案是大小写任意组合都不影响，如上面的 `from` 所示，就算写 `fRom` 或 `FRom` ，都不会影响查询结果。但是在SQL中，通常所有关键字都会使用大写来书写。

```
SELECT staffNo, fname, lname, position FROM Staff
WHERE position IN ('Manager', 'Senior Manager');
```

这里我们的筛选条件是 `position` 列的值必须是“Manager”或者“Senior Manager”。我们使用了 `IN` 关键字来指定多个可能的值，这样就不需要写多个 `OR` 条件了。

```
SELECT staffNo, fname, lname FROM Staff
WHERE lname LIKE '_ll%' OR lname IS NULL;
```

这里我们使用了 `LIKE` 关键字来进行模糊匹配。 `_` 表示 **任意单个字符** ，而 `%` 表示**任意数量的字符（包括零个）** 。所以 `'_ll%'` 就表示姓氏的第二、三位是“ll”，而第一位可以是任意字符，后面可以有任意数量的字符。或者， `lname` 也可以是 `NULL` 。

```
SELECT propertyNo FROM Property
ORDER BY ptype, rent DESC;
```

这里我们使用了 `ORDER BY` 关键字来对结果进行排序。首先按照 `ptype` 列进行默认的升序排序，如果有多个记录的 `ptype` 值相同，那么就按照第二个指定的 `rent` 列进行降序排序。

2.2 Aggregate functions

函数	功能说明	示例
`COUNT`	统计记录数量（参数 `*` ），或统计某列中非空值的数量（参数为列名）	`SELECT COUNT(*) FROM employees;`
`SUM`	计算某个数值列的总和	`SELECT SUM(salary) FROM employees;`
`AVG`	计算某个数值列的平均值	`SELECT AVG(salary) FROM employees;`
`MIN`	获取某列中的最小值	`SELECT MIN(salary) FROM employees;`
`MAX`	获取某列中的最大值	`SELECT MAX(salary) FROM employees;`

```
SELECT COUNT( DISTINCT propertyNo ) AS count FROM Viewing
WHERE viewDate BETWEEN '1-May-22' AND '31-May-22';
```

上述查询会得到一个表头为 `count` ，内容为数字，表示符合筛选条件的数据的数量。其中， `DISTINCT` 关键字确保了只统计不同的 `propertyNo` 的数量。

```
SELECT branchNo,
COUNT( staffNo )AS count,
SUM( salary ) AS sum
FROM Staff GROUP BY branchNo;
```

`GROUP BY` 会首先对所有数据进行分组。值得注意的是， `GROUP BY` 是为聚合函数服务的，所以其子句中的列的信息，都将被唯一化。在上面的例子中，每个 `branchNo` 都会对应一个分组，和输出中的一行， `COUNT(staffNo)` 会统计该分组中 `staffNo` 的数量，而 `SUM(salary)` 会计算该分组中 `salary` 的总和。所以它会得到一个三列，行数等于 `branchNo` 的不同值的数量的结果表。

而 `SELECT` 后如果同时有列名和聚合函数，那么这些列名必须出现在 `GROUP BY` 子句中，否则就会报错。例如，`SELECT staffNo, SUM(salary) FROM Staff;` 是一句非法的语句，因为缺乏 `GROUP BY` 子句来指定如何分组 `staffNo` 。

```
SELECT branchNo, COUNT( staffNo )
AS count, SUM( salary ) AS sum
FROM Staff GROUP BY branchNo
HAVING COUNT( staffNo ) > 1
ORDER BY branchNo LIMIT 0 , 30;
```

`HAVING` 子句是在 `GROUP BY` 之后进行过滤的，它可以使用聚合函数来指定过滤条件。在上面的例子中， `HAVING COUNT(staffNo) > 1` 表示只保留那些 `staffNo` 数量大于1的分组。

最后， `ORDER BY branchNo` 会按照 `branchNo` 的升序对结果进行排序，而 `LIMIT 0, 30` 则表示只返回前30行结果。

2.3 Subqueries

可以使用子查询获得的Result Set来支持上一级的查询。例如：

```
SELECT staffNo, fName, lName, position FROM Staff
WHERE branchNo = ( SELECT branchNo
                   FROM Branch WHERE
                   street = '163 Main St' );
-- 要确保子查询返回的结果是单值，否则会报错

SELECT staffNo, fName, lName, position, salary FROM Staff
WHERE salary > ( SELECT AVG( salary ) FROM Staff );
-- aggregate function不能在WHERE中直接出现

SELECT propertyNo, staffNo FROM Property
WHERE staffNo IN (
    SELECT staffNo FROM Staff
    WHERE branchNo = (
        SELECT branchNo FROM Branch
        WHERE street = '163 Main St'));
-- 如果子查询的结果不确保为单值，使用IN
```

可以使用 `SOME` ， `ALL` 和 `EXISTS` 来执行一些一阶逻辑。例如：

```
SELECT staffNo, fName, lName, salary FROM Staff
WHERE salary > SOME (
    SELECT salary FROM Staff
    WHERE branchNo = 'B003');
-- 只要子查询结果中有一个值满足条件，就判定为TRUE

SELECT staffNo, fName, lName, salary FROM Staff
WHERE salary > ALL (
    SELECT salary FROM Staff
    WHERE branchNo = 'B003');
-- 只有当子查询结果中的所有值都满足条件时，才判定为TRUE
-- 和通常的for all一样，如果子查询结果为空，那么ALL条件会被判定为TRUE

SELECT staffNo, fName, lName, position
FROM Staff s -- 别名，和Staff AS s等价，父查询中的表名可以在子查询中使用
WHERE EXISTS (
    SELECT * FROM Branch b
    WHERE s.branchNo = b.branchNo AND city = 'London');
-- 对于s，如果存在一个b满足条件，那么EXISTS就检测到不为空，返回TRUE
-- NOT EXISTS则是当子查询结果为空时返回TRUE
```

`FROM` 后面可以有多个表，通过这种方法可以实现表的连接。例如：

```
SELECT Client.clientNo, propertyNo, comment
FROM Client, Viewing
WHERE Client.clientNo = Viewing.clientNo;

SELECT c.clientNo, propertyNo, comment
FROM Client c, Viewing v
WHERE c.clientNo = v.clientNo;
-- 这两者等价，第二个使用了别名来简化表名的书写
```

对于上面的查询，以下面的信息为例

clientNo	fName	lName
C001	Alice	Zhang
C003	Cathy	Wang

clientNo	propertyNo	viewDate	comment
C001	P101	2026-03-20	very good
C001	P102	2026-03-21	too expensive
C002	P103	2026-03-21	nice location

查询结果为

clientNo	propertyNo	comment
C001	P101	very good

clientNo	propertyNo	comment
C001	P102	too expensive

2.4 JOIN Queries

`JOIN` 有两种基础语法，一种是 `FROM ... JOIN ... ON` 另一种是 `FROM ... JOIN ... USING`

```
SELECT ...
FROM Clients AS c
JOIN Viewings AS v ON c.clientNo = v.clientNo;

SELECT ...
FROM Client
JOIN Viewing USING (clientNo);
```

可以看到 `USING` 更加简洁，也只能指定在两张表中的相同列明进行连接，而 `ON` 除了实现这个功能以外，自然还可以指定其他条件。 `JOIN` 的默认类型是 `INNER JOIN` ，它只返回两张表中满足连接条件的行。而使用

```
FROM Branch b [LEFT | RIGHT | FULL]
JOIN Property p ON b.bCity = p.pCity;
```

可以实现外连接（Outer Join）。其中， `LEFT JOIN` 会返回左表（Branch）中的所有行，即使它们在右表（Property）中没有匹配的行； `RIGHT JOIN` 则相反，会返回右表中的所有行； `FULL JOIN` 会返回两张表中的所有行，无论是否有匹配的行。例如

```
SELECT b.*, p.*
FROM Branch b
LEFT JOIN Property p ON b.bCity = p.pCity;
```

会返回所有Branch表中的行，如果某个Branch表中的行在Property表中没有匹配的行，那么对应的Property表中的列会被填充为NULL。

如果我们只是想要对特定列的结果进行连接，可以使用集合操作符 `UNION` 、 `INTERSECT` 和 `EXCEPT` 来实现。与此同时，这些操作符也会去除重复的值，如果想要保留重复的行，可以使用 `UNION ALL` 、 `INTERSECT ALL` 和 `EXCEPT ALL` 。

3.1 Relational Algebra

在开始讲实际运算方法之前，我们需要了解这个领域使用的运算符。它被称为关系代数（Relational Algebra）。

- σ（**Selection**）：选择特定行。例如，$\sigma_{age>30}(Employees)$ 表示从 `Employees` 关系中选择年龄大于30的员工。
- π（**Projection**）：选择特定列。例如，$\pi_{name,age}(Employees)$ 表示从 `Employees` 关系中选择 `name` 和 `age` 列。
- $\cup$（**Union**）：合并两个关系中的行。例如，$Employees \cup Managers$ 表示将 `Employees` 和 `Managers` 关系中的所有行合并在一起。
- $-$（**Difference**）：从一个关系中去除另一个关系中的行。例如，$Employees - Managers$ 表示从 `Employees` 关系中去除那些也在 `Managers` 关系中的行。
- $\cap$（**Intersection**）：选择两个关系中都存在的行。例如，$Employees \cap Managers$ 表示选择那些既在 `Employees` 关系中又在 `Managers` 关系中的行。
- $\times$（**Cartesian Product**）：将两个关系中的每一行组合在一起。例如，$Employees \times Departments$ 表示将 `Employees` 和 `Departments` 关系中的每一行组合在一起。也就是说，结果的行数是两个关系中行数的乘积。

乘积很适合与选择操作结合使用。例如，

$$\sigma_{Employees.deptId=Departments.deptId}(Employees \times Departments)$$

表示先将 `Employees` 和 `Departments` 关系中的每一行组合在一起，然后选择那些 `deptId` 列值相等的行。

- $\bowtie$（**Theta Join**）：根据某个条件将两个关系中的行组合在一起。例如，$Employees \bowtie_{Employees.deptId=Departments.deptId} Departments$ 表示将 `Employees` 和 `Departments` 关系中的行根据 `deptId` 列进行连接。

注意到，其语义和上面的乘积加选择是一样的。也就是说

$$R \bowtie_F S = \sigma_F(R \times S)$$

- $\bowtie$（**Natural Join**）：将两个关系中的行根据所有同名列进行连接。例如，如果 `Employees` 和 `Departments` 的相同列只有 `deptId` ，那么 $Employees \bowtie Departments$ 表示将两个关系中的行根据 `deptId` 列的相同性进行连接，与上面的例子语义相同。
- $\bowtie$ （**Outer Join**）：在连接的基础上保留某个关系中的所有行。例如，$Employees \bowtie Departments$ 表示将 `Employees` 和 `Departments` 进行连接，并且保留 `Employees` 中的所有行，即使它们在 `Departments` 中没有匹配的行。相比Natural Join，Outer Join的结果中会多一些无法匹配项被填充为 `NULL` 的行。
- $\div$（**Division**）：返回被查询列中完全包含查询条件的元素。例如，如果我们有一个 `Students` 关系和一个 `Courses` 关系，并且我们想要找到那些选修了所有课程的学生，我们可以使用除法操作来实现。例如，$Students \div Courses$ 表示找到那些选修了 `Courses` 关系中所有课程的学生。
- **Aggregate Operations**：如 `SUM` 、 `AVG` 、 `COUNT` 等聚合函数也可以被视为关系代数中的操作，用于对关系中的数据进行汇总和统计。

3.2 Query Processing

我们以下面这个查询为例：

```
SELECT *
FROM Staff s, Branch b
WHERE s.branchNo = b.branchNo AND
(s.position = 'Manager' AND b.city = 'London');
```

假设 `Staff` 和 `Branch` 的行数分别为 1000 和 50，Manager和London的数量分别为 50 和 5，并假设所有中间结果也会同等地被储存，这段查询对应的读写开销有以下三种可能：

Cartesian product followed by selection:

$$\sigma_{position='Manager' \wedge city='London' \wedge Staff.branchNo=Branch.branchNo}(Staff \times Branch)$$

在这种方法中，会读取所有数据然后进行笛卡尔积，得到 $1000 \times 50 = 50000$ 行的中间结果，然后再进行选择操作来过滤出满足条件的行。总的读写开销是 $1000 + 50 + 50000 + 50000 = 101050$。

Join followed by selection:

$$\sigma_{position='Manager' \wedge city='London'}(Staff \bowtie_{Staff.branchNo=Branch.branchNo} Branch)$$

在这种方法中，首先进行连接操作，需要读取所有数据，而得到的结果数量等于总员工数量，即 1000 行。然后再进行选择操作来过滤出满足条件的行。总的读写开销是 $1000 + 50 + 1000 + 1000 = 3050$。

Selections followed by join:

$$(\sigma_{position='Manager'}(Staff)) \bowtie_{Staff.branchNo=Branch.branchNo} (\sigma_{city='London'}(Branch))$$

首先看连接左侧，通过读取 1000 行数据，筛选出所有Manager，得到 50 行数据。

右侧通过读取 50 行数据，筛选出所有London，得到 5 行数据。

最后对两侧得到的数据进行Theta Join，只需要读取上一步得到的所有结果。总的读写开销是 $1000 + 50 + (50 + 5) \times 2 = 1160$。

总结一下，虽然

$$R \bowtie_F S = \sigma_F(R \times S)$$

存在于语义上。但是在实际查询过程中，使用join操作通常会比先进行笛卡尔积再进行选择更高效，因为它可以避免生成大量的中间结果。

4 Designing Theories

4.1 Transaction Management

在节中，我们介绍了事务的四个特性：原子性（Atomicity）、一致性（Consistency）、隔离性（Isolation）和持久性（Durability），简称ACID。它们指的是：

- **原子性**：‘All or nothing’ property.
- **一致性**：Must transform database from one consistent state to another.
- **隔离性**：Partial effects of incomplete transactions should not be visible to other transactions.
- **持久性**：Effects of a committed transaction are permanent and must not be lost because of later failure.

那么在错误的设计中，就会因为并发处理问题，而产生对上述性质的违背。简单来说，在没有正确的lock的情况下，当两个事务同时访问同一数据时，就可能出现读写的混乱。

但是，在一些情况下，过多的lock可能会导致死锁。死锁是指两个或多个事务在等待对方释放资源，从而导致所有事务都无法继续执行的情况。例如，事务A持有资源X并等待资源Y，而事务B持有资源Y并等待资源X，这样就形成了一个循环等待的情况，导致死锁。

4.2 Fragment

在分布式数据库中，数据可以被分为不同的片段（Fragment），每个片段可以存储在不同的节点上。分片可以基于不同的策略进行，例如：

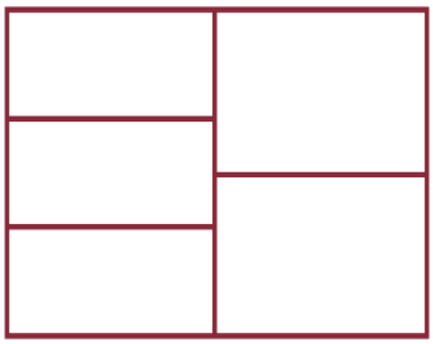
1. **水平分片（Horizontal Fragmentation）**：将表中的行分成不同的片段，所有片段有相同的列。

```
Student_London = SELECT *
FROM Student
WHERE campus = 'London';
```

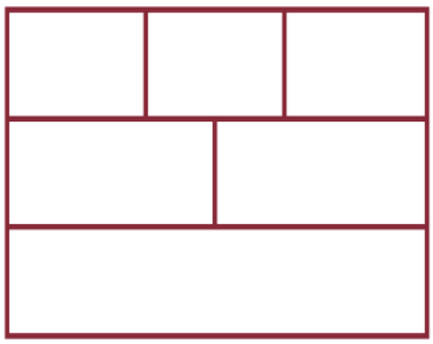
2. **垂直分片（Vertical Fragmentation）**：将表中的列分成不同的片段，所有片段通过主键连接。

```
Student_Basic(sid, name, campus)
Student_Academic(sid, major, age)
```

3. **混合分片（Mixed Fragmentation）**：结合水平和垂直分片的策略。



(a)



(b)

4.3 Data Warehouse

A subject-oriented, integrated, time-variant, and non-volatile collection of data in support of management's decision-making process.

其与下一节要讲的OLAP的区别如下

CHARACTERISTIC	OLTP SYSTEMS	DATA WAREHOUSING SYSTEMS
Main purpose	Support operational processing	Support analytical processing
Data age	Current	Historic (but trend is toward also including current data)
Data latency	Real-time	Depends on length of cycle for data supplements to warehouse (but trend is toward real-time supplements)
Data granularity	Detailed data	Detailed data, lightly and highly summarized data
Data processing	Predictable pattern of data insertions, deletions, updates, and queries. High level of transaction throughput.	Less predictable pattern of data queries; medium to low level of transaction throughput
Reporting	Predictable, one-dimensional, relatively static fixed reporting	Unpredictable, multidimensional, dynamic reporting
Users	Serves large number of operational users	Serves lower number of managerial users (but trend is also toward supporting analytical requirements of operational users)

简单来说，OLTP 处理当前、实时、细粒度、高频交易数据；数据仓库处理历史数据、汇总数据、多维分析查询，查询模式更不可预测。

其大致模式如下：

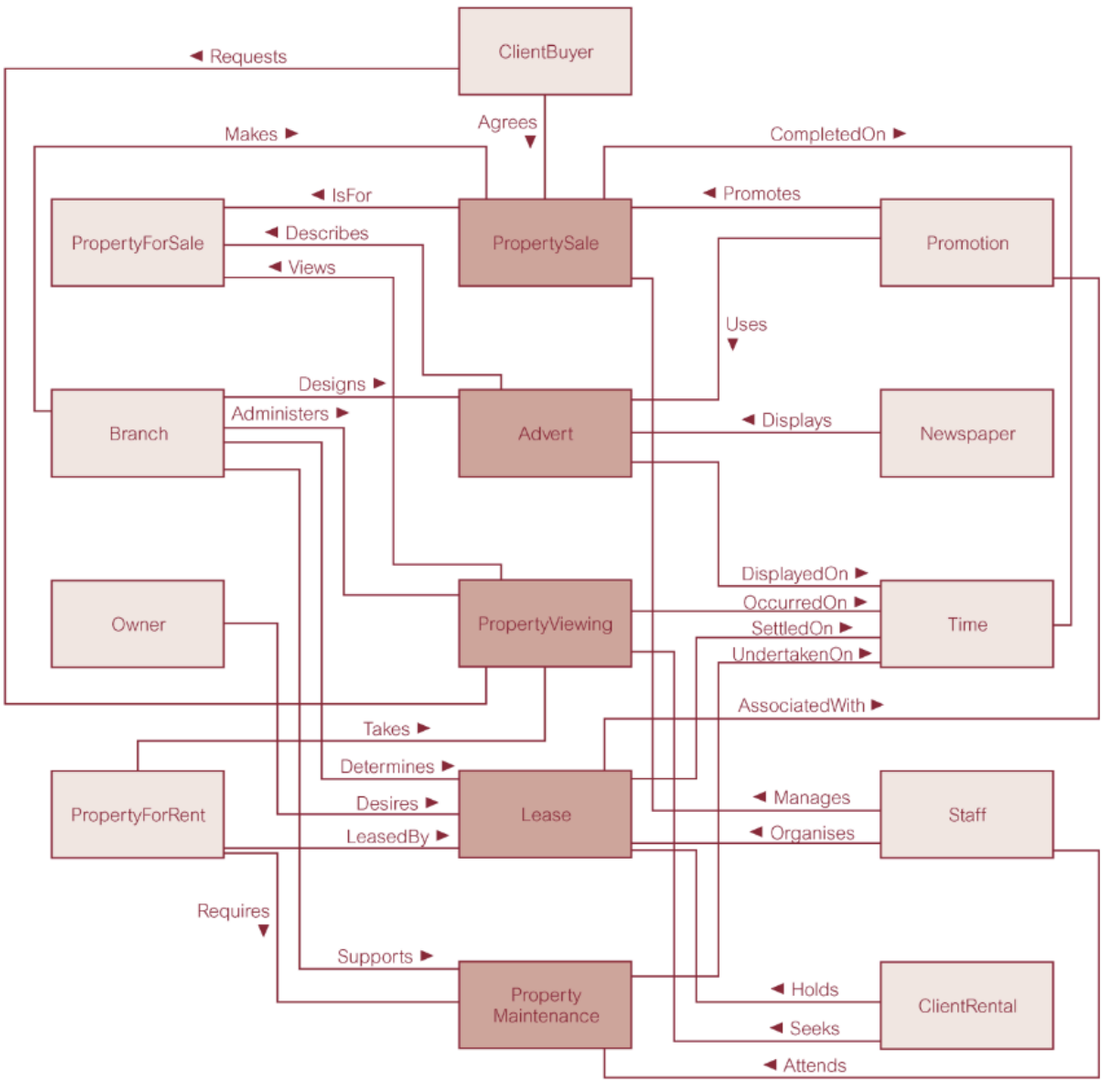
数据源 → ETL → 仓库(DBMS) → 查询/分析工具(via Query Manager)

这里的ETL指的是Extract（提取）、Transform（转换）和Load（加载）。它是将数据从源系统提取出来，进行清洗、转换和整合，然后加载到数据仓库中的过程。其工作的目的是提高数据质量。

在DBMS中，数据仓库通常使用专门的架构来优化查询性能，例如星型模式（Star Schema）和雪花模式（Snowflake Schema）。在星型模式中，中心是一个事实表（Fact Table），周围是多个维度表（Dimension Tables）。在雪花模式中，维度表被进一步规范化成多个相关的表。

构建数据结构时，可以考虑 **Kimball's Business Dimensional Lifecycle**：先定义业务过程（Business Process），然后定义粒度（Grain），接着设计维度（Dimensions）和事实（Facts），最后构建数据模型（Data Model）。

以下就是一个经过设计的、以事实为中心的例子



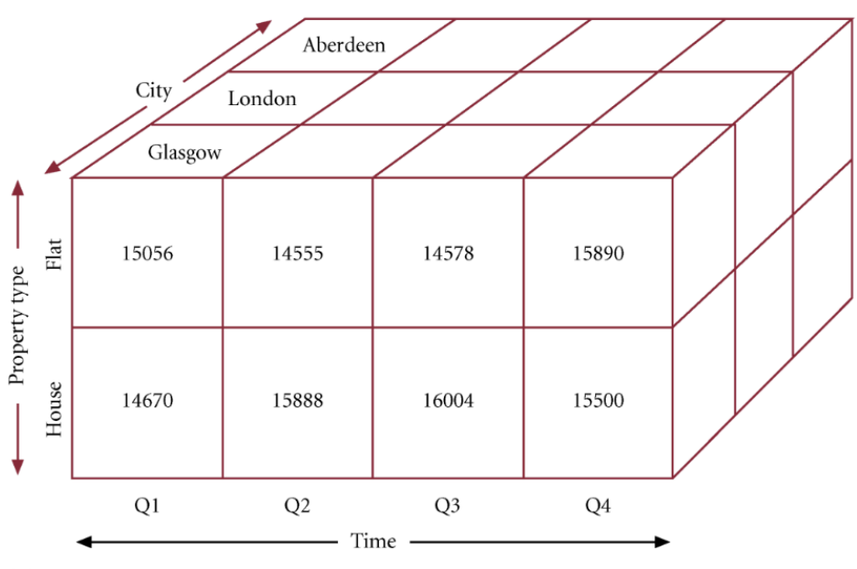
4.4 OLAP

在线分析处理（OLAP）是一种用于支持复杂查询和多维分析的技术。它允许用户从不同的角度和层次对数据进行分析，以便更好地理解业务情况和做出决策。其三个特征是：

1. Multi-dimensional views of data：多维视图
2. Support for complex calculations：支持复杂计算
3. Time intelligence：时间智能，例如 year-to-date、period-over-period comparison 等。

举个例子，可以按下图所示的方式对数据进行多维分析：

Property Type	City	Time	Total Revenue
Flat	Glasgow	Q1	15056
House	Glasgow	Q1	14670
Flat	Glasgow	Q2	14555
House	Glasgow	Q2	15888
Flat	Glasgow	Q3	14578
House	Glasgow	Q3	16004
Flat	Glasgow	Q4	15890
House	Glasgow	Q4	15500
Flat	London	Q1	19678
House	London	Q1	23877
Flat	London	Q2	19567
House	London	Q2	28677
.....	.....	.....	.....
.....	.....	.....	.....



与我们在 1.3 讲的数据清洗在操作上有些类似，OLAP会在数据仓库中进行数据的清洗和转换，以便更好地支持分析查询。例如，可以使用OLAP工具来创建聚合表（Aggregate Tables），这些表预先计算了某些维度的汇总数据，从而加快查询速度。

4.5 Semi-structured data

使用XML和JSON等格式来存储半结构化数据。它们允许数据以层次化的方式组织，并且可以包含不同类型的数据。例如，下面是一个XML格式的学生信息：

```
<student>
  <name>Alice</name>
  <email>alice@example.com</email>
  <address>
    <city>London</city>
    <postcode>WC1E</postcode>
  </address>
</student>
```

这样的数据通常通过 **SQL/XML Mapping Functions** 产生，例如 `XMLELEMENT`、 `XMLATTRIBUTES` 和 `XMLAGG` 等函数来生成XML数据。而使用 **XQuery** 可以对XML数据进行查询和操作，例如：

```
for $s in //student
where $s/address/city = 'London'
return $s
```

和SQL语句相同，只不过XQuery是用来查询XML数据的。

XML的编写基于某种规则，通常这种规则使用 **Document Type Definitions (DTD)** 或 **XML Schema** 来定义。而一个在XML语法下合法的文档被称为 **well-formed XML**，而在此基础上符合特定DTD或XML Schema的文档被称为 **valid XML**。

5 Security

5.1 Forms and Data Validation

5.1.1 SQL Injection

SQL注入是一种攻击方式，攻击者通过在输入中插入恶意的SQL代码来操纵数据库。例如，如果一个登录表单没有正确地处理用户输入，攻击者可能会输入以下内容：

```
x'; DROP TABLE members; --
```

如果应用程序直接将这个输入插入到SQL查询中，可能会执行以下查询：

SELECT email, passwd, login_id FROM members
WHERE email = '**x'; DROP TABLE members; --';**

如果你还不明白的话，我们放到SQL语句中来看看：

```
SELECT email, passwd, login_id FROM members
WHERE email = 'x'; DROP TABLE members; --';
```

在这个查询中， `x'` 结束了原本的字符串输入，然后 `DROP TABLE members;` 是一个新的SQL命令，用于删除 `members` 表，而 `--` 是SQL中的注释符号，表示后面的内容将被忽略。因此，这个输入会导致数据库执行删除表的命令，从而造成数据损失。

5.1.2 Cross-site Scripting (XSS)

跨站脚本攻击（XSS）是一种安全漏洞，攻击者通过在网页中注入恶意的JavaScript代码来窃取用户信息或执行其他恶意操作。例如，如果一个评论表单没有正确地处理用户输入，攻击者可能会输入以下内容：

```
<script>alert('XSS Attack!');</script>
```

那么它会在其他用户的浏览器被自动执行，造成非预期的行为。

5.1.3 Validation Strategies

1. **Input Validation**：对用户输入进行严格的验证，确保其符合预期的格式和内容。例如，可以使用正则表达式来验证电子邮件地址的格式，或者限制输入的长度和字符类型。
2. **Parameterized Queries**：使用参数化查询（Prepared Statements）来防止SQL注入攻击。参数化查询将用户输入作为参数传递给SQL语句，而不是直接将其拼接到SQL字符串中，从而避免了恶意代码的执行。
3. **Output Encoding**：对输出进行编码，防止跨站脚本攻击。例如，在将用户输入显示在网页上时，可以使用HTML实体编码来转义特殊字符（如 `<` 被编码为 `<`），防止它们被解释为HTML或JavaScript代码。

5.2 Access Control

5.2.1 Passwords

不要明文储存密码。对密码进行哈希处理，并且使用盐（salt）来增加安全性。在MySQL中，可以使用 `SHA` 函数来对密码进行哈希处理，例如：

```
INSERT INTO Users (username, password) VALUES
('someone', SHA('password'))
```

它是一个返回 `VARCHAR(40)` （160bit）的函数，表示输入字符串的SHA-1哈希值。

现在已经不建议使用SHA-1了，推荐使用更安全的哈希算法，如SHA-256或bcrypt等。

5.2.2 Cookies

Cookies是存储在用户浏览器中的小文本文件，用于保存用户的登录状态和其他信息。由于HTTP的无状态特性，Cookies成为了一种常用的会话管理机制。浏览器会在每一个请求中自动发送相关的Cookies，这样服务器就可以识别用户的身份并保持会话状态。

Cookies的内容通常包括用户名、过期时间等信息。与此同时，用于记录会话状态的Cookies通常会包含一个唯一的会话ID（Session ID），服务器通过这个ID来识别用户的会话。

要注意的是，Cookies是储存在用户浏览器中的，因此它们可能会被用户修改或盗用。为了提高安全性，可以使用以下措施：

1. **Secure Flag**：设置Cookies的Secure标志，确保它们只能通过HTTPS连接发送，从而防止被窃取。
2. **HttpOnly Flag**：设置Cookies的HttpOnly标志，防止JavaScript访问Cookies，从而减少跨站脚本攻击的风险。
3. **SameSite Attribute**：设置Cookies的SameSite属性，限制Cookies在跨站请求中的发送，从而防止跨站请求伪造（CSRF）攻击。